Паралельне сортування підрахунком

Варіант 3 · Основна реалізація на Python

Мета. Розробити паралельний алгоритм сортування підрахунком і визначити, як його час залежить від кількості елементів, кількості працівників та діапазону значень.

Основна ідея. Кожен працівник отримує частину списку та рахує, скільки разів у ній зустрічається кожне число. Головна програма додає ці частоти й записує числа за зростанням.

1 Вхідні дані та позначення

Позначення	Значення
A	Вхідний список цілих чисел
n	Кількість елементів списку
p	Кількість робочих процесів у Python
lo	Найменше дозволене значення ключа
K	Кількість дозволених значень у діапазоні

Дозволені значення: від lo до lo + K - 1. Наприклад, для діапазону від -1 до 4 маємо lo = -1 та K = 6. Для від’ємних чисел індекс лічильника обчислюється як x - lo.

2 Приклад сортування

Потрібно впорядкувати список **[4, -1, 2, 4, 0, -1, 3, 2]**. Двом працівникам передаємо по чотири елементи.

Ключ	-1	0	1	2	3	4
Частоти у [4, -1, 2, 4]	1	0	0	1	0	2
Частоти у [0, -1, 3, 2]	1	1	0	1	1	0
Загальні частоти	2	1	0	2	1	2

Читаємо останній рядок зліва направо: дві одиниці зі знаком мінус, один нуль, жодної одиниці, дві двійки, одна трійка та дві четвірки. Отримуємо **[-1, -1, 0, 2, 2, 3, 4, 4]**.

Умова застосування. Метод доцільний для помірного K. Якщо можливих ключів дуже багато, таблиця частот стає надто великою. У цій роботі сортуються цілі числа без пов’язаних записів.

3 Як організовано паралельний алгоритм

Розбиття. Список ділиться на p неперетинних блоків приблизно однакового розміру. Для працівника з номером t початок і кінець блоку задаються формулами:

$$\ell_t = \left\lfloor \frac{nt}{p} \right\rfloor, \qquad r_t = \left\lfloor \frac{n(t+1)}{p} \right\rfloor, \qquad t = 0, \ldots, p-1$$

Працівник обробляє індекси від ℓ_t включно до r_t невключно. Позначення ⌊…⌋ означає округлення вниз. Наприклад, при n = 8 і p = 2 це блоки з індексами [0, 4) та [4, 8).

Підрахунок. Працівники незалежно заповнюють свої таблиці частот. **Очікування.** Головна програма чекає, доки завершаться всі підрахунки та будуть отримані всі таблиці. Лише після цього можна обчислити загальні частоти.

Об’єднання. Для кожного ключа додаємо частоти з усіх блоків:

$$H[v] = \sum_{t=0}^{p-1} H_t[v], \qquad v = 0, \ldots, K-1$$

Тут $H_t[v]$ - кількість ключів lo + v у блоці t, а H[v] - їх кількість у всьому списку. Для прикладу на першій сторінці частота ключа -1 дорівнює 1 + 1 = 2.

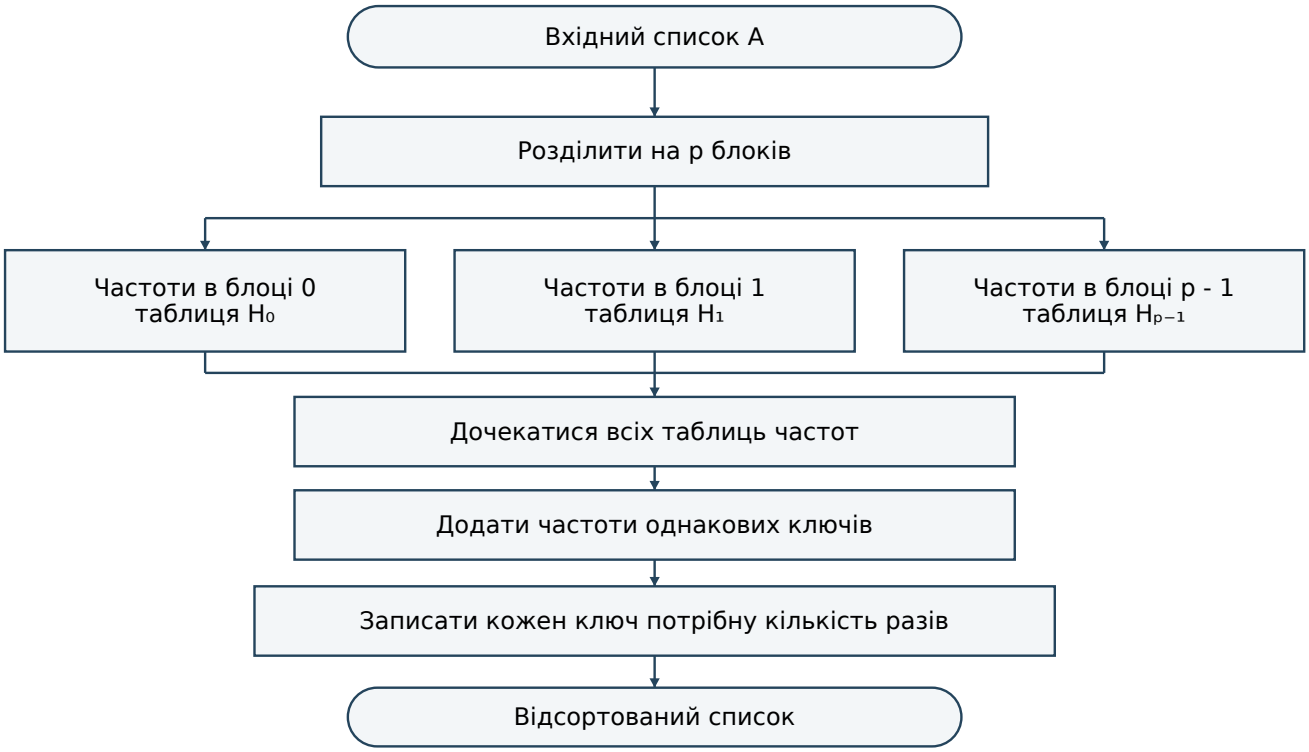


Рисунок 1. Підрахунки в середньому рядку незалежні й можуть виконуватися одночасно. Після отримання всіх таблиць об’єднання та відновлення виконує головна програма. Для p = 2 використовуються лише два блоки.

У Python працівники є окремими процесами. Кожен отримує копію свого блоку і змінює лише власну таблицю. У звіті описано дії алгоритму; назви функцій і налаштування запуску наведено в ноутбуці.

4 Перевірка результатів

Вихідні списки послідовної та паралельної реалізацій порівнювали з результатом стандартної функції sorted у Python. Перевірка полягала у поелементному порівнянні списків однакової довжини. Її виконували після зупинки таймера.

Приклади очікуваних результатів

Вхідний список	Очікуваний результат	Випадок
[]	[]	Порожній список
[5]	[5]	Один елемент
[2, 1, 2]	[1, 2, 2]	Повтори
[-1, 2, -1]	[-1, -1, 2]	Від’ємні числа
[3, 2, 1]	[1, 2, 3]	Зворотний порядок
[2, 1], p = 3	[1, 2]	Працівників більше, ніж елементів

Результати перевірки. Для всіх 80 тестових входів обидві Python-реалізації повернули списки, що повністю збіглися з результатами sorted. У 6 додаткових перевірках нецілі ключі та значення за межами дозволеного діапазону були відхилені. Усі результати вимірювань також збіглися з еталоном.

5 Складність та показники ефективності

Послідовна версія рахує n елементів, переглядає K лічильників і записує n елементів виходу:

$$T_{\text{seq}} = \Theta(n + K)$$

У паралельній версії підрахунок одного блоку має порядок n/p + K. Однак створення копій блоків, об’єднання p таблиць і запис результату залишаються послідовними. Додаються витрати передавання даних між процесами.

$$T_p \approx c_1 \frac{n}{p} + c_2 pK + c_3 n + C_{\text{IPC}}(n, pK)$$

$$S_p = \frac{T_{\text{seq}}}{T_p}, \qquad E_p = \frac{S_p}{p}$$

S_p показує прискорення, E_p - ефективність відносно кількості працівників. При $S_p < 1$ паралельний запуск повільніший. Через послідовні етапи збільшення p не гарантує зменшення часу.

6 Методика та результати вимірювання

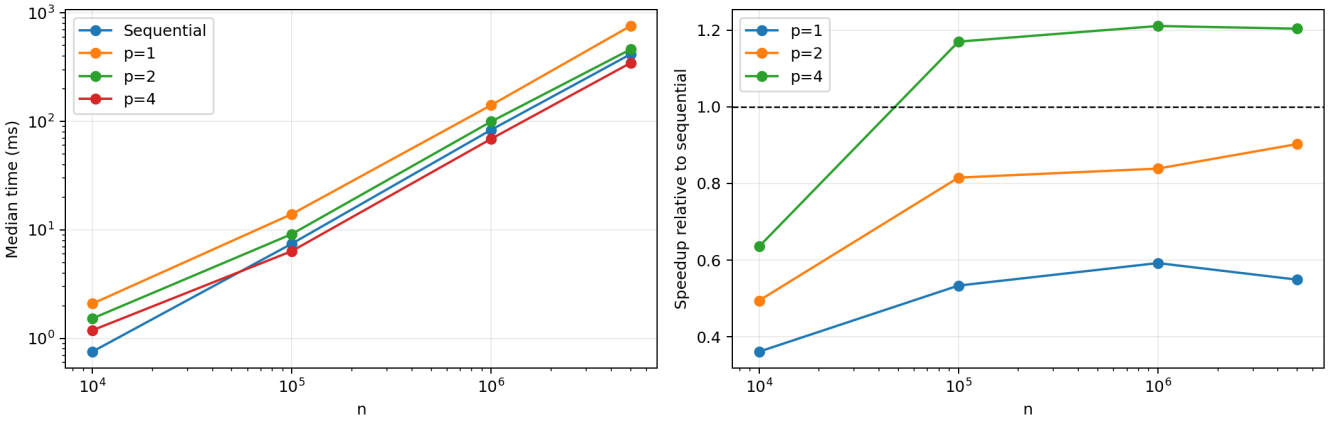
Основна Python-версія використовує multiprocessing, без OpenMP і спільного масиву. Для кожної конфігурації виконано один прогрів і п’ять вимірювань. У таблицях наведено медіани. Всі методи отримують однакові дані з фіксованим seed 42.

Вимірюється створення зрізів, передавання блоків, підрахунок, повернення й додавання частот та створення виходу. Пул процесів уже запущений. Його запуск виміряно окремо. Генерація даних, еталонне сортування, перевірка й графіки не входять у час.

Середовище. Linux x86_64, Python 3.12.14, AMD EPYC 9V74. Видно 9 логічних CPU, ліміт CPU-потужності - 8.

Різні розміри списку за K = 256

n	Tseq мс	Tp p=1 мс	Tp p=2 мс	Tp p=4 мс	S_4
10 000	0.75	2.09	1.53	1.19	0.64
100 000	7.46	13.98	9.15	6.37	1.17
1 000 000	83.52	141.01	99.57	68.97	1.21
5 000 000	415.45	756.68	460.16	345.10	1.20



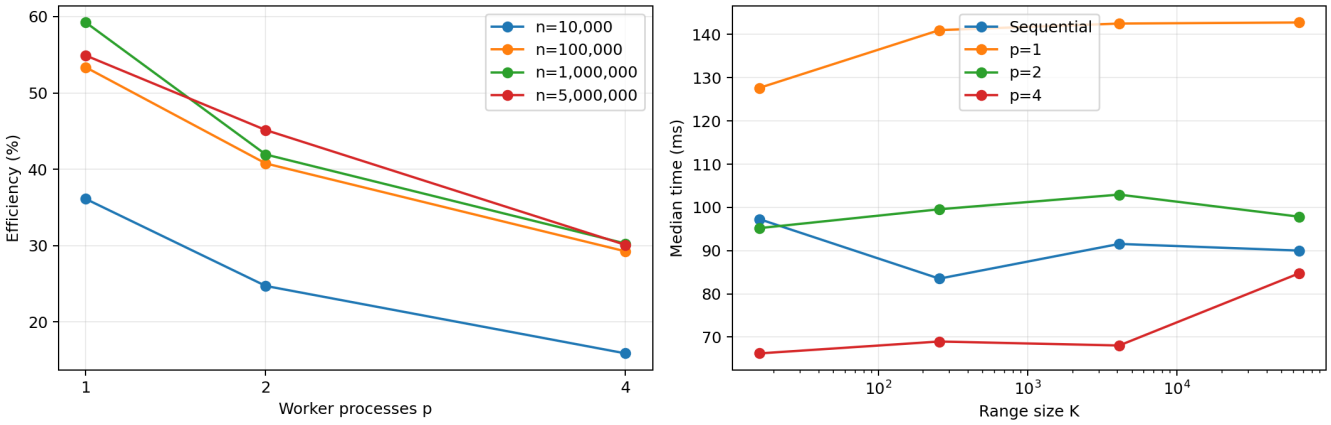
Для 10 000 елементів S_4 = 0.64: витрати паралельного запуску переважають. Для 5 000 000 елементів час зменшився з 415.45 до 345.10 мс, тобто S_4 = 1.20, E_4 = 30.1%.

Один працівник у пулі не тотожний послідовному запуску: він усе одно отримує блок через міжпроцесне передавання. Тому конфігурація p = 1 дозволяє побачити накладні витрати без виграшу від розподілу підрахунку.

7 Вплив діапазону та накладних витрат

Фіксовано n = 1 000 000. Збільшення K означає більші таблиці в кожного працівника, більше даних для повернення і більше додавань під час об’єднання.

K	Tseq мс	Tp p=2 мс	Tp p=4 мс	S_4
16	97.28	95.20	66.22	1.47
256	83.52	99.57	68.97	1.21
4 096	91.55	102.96	68.05	1.35
65 536	90.01	97.86	84.74	1.06



Час не зобов’язаний зростати монотонно разом із K. На нього також впливають розподіл ключів, робота з об’єктами Python, пам’ять і планувальник. Таблиця показує фактичний результат, а теоретична оцінка пояснює складові витрат.

Запуск пулу процесів виміряно окремо

Кількість працівників	Час запуску мс
1	69.54
2	74.43
4	70.89

Для одноразового сортування до часу потрібно додати запуск працівників. При повторному використанні пулу він створюється один раз на серію задач. Саме тому ці два режими розрізняються в порівнянні чотирьох мов.

Висновок основного експерименту

Незалежний підрахунок частот вдалося виконати паралельно. Помітного лінійного прискорення немає: копіювання, передавання та послідовні етапи забирають значну частину часу. Для малих списків послідовна версія краща; для більших у частині конфігурацій отримано помірний виграш.

8 Порівняння п’яти мов

Додатковий експеримент порівнює конкретні реалізації того самого алгоритму. У кожній версії спочатку створюються приватні копії p блоків, потім паралельно обчислюються частоти. Об’єднання таблиць і запис результату послідовні.

Мова	Працівники	Дані задачі
Python	Окремі процеси	Копія списку, серіалізація і IPC
C++	Пул стандартних потоків	Окрема копія вектора int32
Rust	Пул стандартних потоків	Власний Vec із ключами int32
Go	Горутини	Окрема копія slice із ключами int32
Node.js	worker_threads	Копії Int32Array, передані буфери

У цьому порівнянні OpenMP не застосовується. У скомпільованих версіях працівники працюють в одному адресному просторі, але кожна задача має власний блок і таблицю. У Python блок передається в інший процес. Тому результати залежать і від мови, і від моделі виконання.

Два режими вимірювання

Warm. Пул працівників уже створено. Час включає копії блоків, передавання задач, підрахунок, повернення таблиць, їх об’єднання та відновлення виходу.
Cold. Додатково включено створення і завершення працівників. Старт головного інтерпретатора або виконуваного файла не входить у цей час. У Go горутини можуть використовувати вже наявні потоки ОС.

Однакові вхідні дані

Для кожного сценарію створюється один файл із ключами у форматі little-endian int32. Саме цей файл читають усі п’ять програм. Його розмір та SHA-256 збережено в inputs.json. Генерація та читання файлу виконуються поза таймером.

Параметри: n = 10 000, 100 000, 1 000 000, 10 000 000; K = 256 та 65 536; p = 1, 2, 4. Для кожної мови є послідовна база та по три конфігурації warm і cold. По п’ять повторів дають 1400 вимірювань. Кожен вихід цілком звірено з еталонним сортуванням.

Node.js передає власність на копії буферів, без SharedArrayBuffer. Лічильники Float64Array точно представляють усі частоти в цих тестах. Прогрів Node.js: 10 запусків seq/warm, один cold; у решті версій один прогрів на конфігурацію.

Версії та збірка

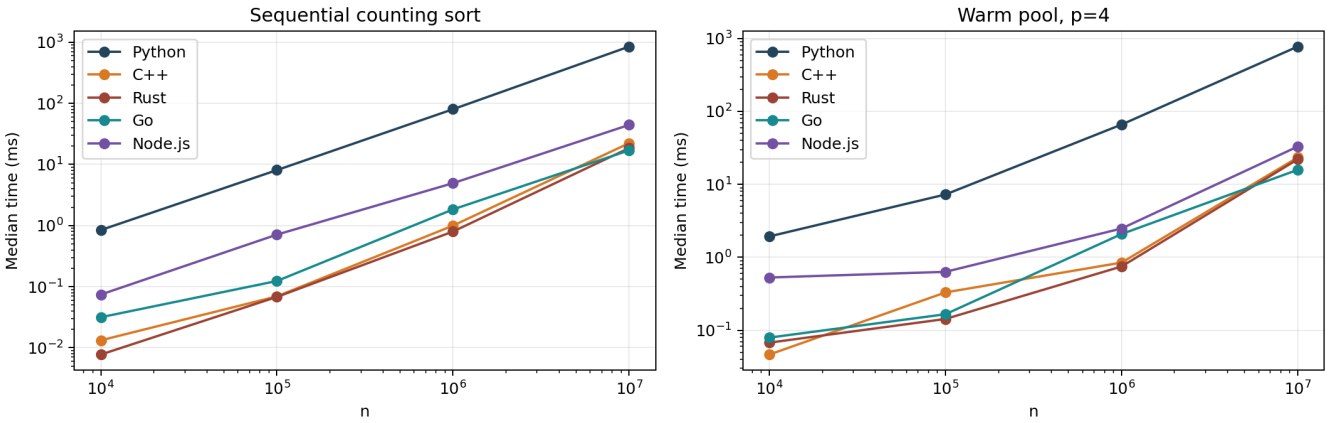
Мова	Версія	Налаштування
Python	3.12.14	CPython, spawn
C++	g++ 13.3.0	-O3 -std=c++17 -pthread
Rust	rustc 1.85.0 (4d91de4e4 2025-02-17)	-O --edition=2021
Go	go version go1.24.0 linux/amd64	go build, стандартний GC
Node.js	v24.19.0	V8, стандартний GC

Усі результати отримано послідовно на тій самій машині. CPU-потужність за квотою: 8; GOMAXPROCS = 8.

9 Абсолютний час виконання

Найбільший список: n = 10 000 000, K = 256. Час у мілісекундах, медіана п’яти повторів. S_4 порівнюється з послідовною версією тієї самої мови.

Мова	Послідовно	Warm p=4	Cold p=4	S_4 warm	E_4 warm
Python	850.072	773.255	959.263	1.10	27.5%
C++	22.313	23.326	24.864	0.96	23.9%
Rust	18.539	22.018	19.097	0.84	21.1%
Go	17.008	15.825	17.759	1.07	26.9%
Node.js	44.427	32.983	99.290	1.35	33.7%



Серед конфігурацій warm з чотирма працівниками для цього входу найменший час отримано у Go: 15.825 мс. Це результат конкретних програм, компіляторів і поточної машини.

Python використовує об’єкти int та міжпроцесне передавання. Скомпільовані версії працюють із компактними масивами та потоками або горутинами. Тому різницю абсолютного часу не можна приписати тільки якості розпаралелювання.

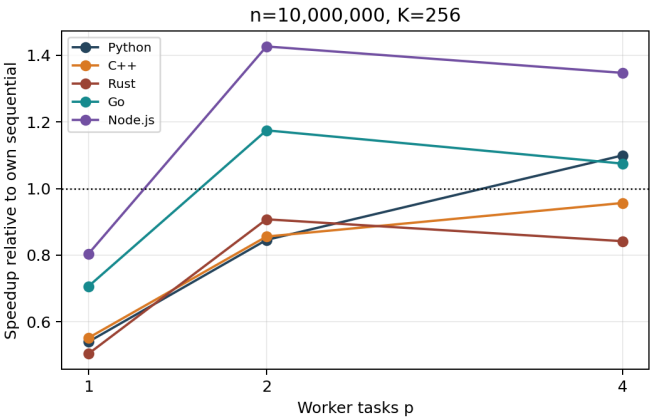
Ширший діапазон ключів

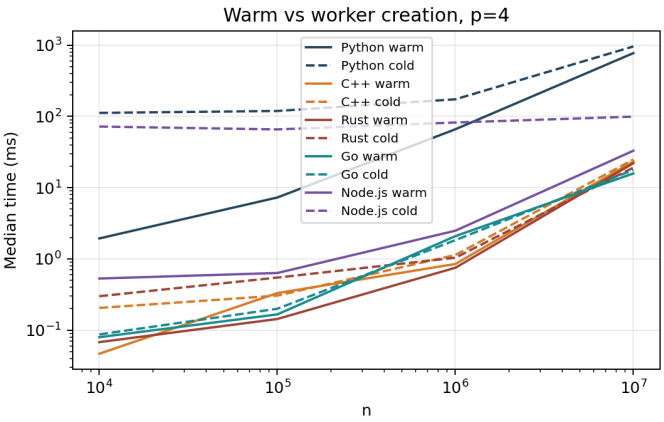
Для n = 10 000 000 та K = 65 536 таблиці частот і їх об’єднання стають більшими. Вхід знову однаковий для всіх мов.

Мова	Послідовно мс	Warm p=4 мс	S_4 warm
Python	893.022	755.173	1.18
C++	26.165	24.954	1.05
Rust	23.753	20.485	1.16
Go	19.881	17.716	1.12
Node.js	45.991	38.543	1.19

Більше K не гарантує монотонного зростання виміряного часу: важливі також доступ до пам’яті, розподіл ключів, робота стандартних бібліотек і розкид запусків.

10 Масштабування та висновки





Ліворуч кожна мова має власну послідовну базу. Праворуч суцільні лінії відповідають warm, пунктирні - cold. Створення горутин, потоків та процесів має різну вартість, тому режими показано окремо.

Мова	S_1 warm	S_2 warm	S_4 warm
Python	0.54	0.85	1.10
C++	0.55	0.86	0.96
Rust	0.50	0.91	0.84
Go	0.71	1.17	1.07
Node.js	0.80	1.43	1.35

Що встановлено

Для найбільшого списку за K = 256 найбільше прискорення серед перевірених warm-конфігурацій становило 1.43 у Node.js при p = 2. Найбільше прискорення та найменший абсолютний час - різні критерії.

Збільшення кількості працівників не усуває послідовного копіювання, додавання таблиць та запису результату. На малих входах витрати диспетчеризації або створення працівників можуть перевищувати виграш від паралельного підрахунку.

Експеримент не визначає універсально найшвидшу мову. На час впливають представлення даних, алокації, компілятор, пул працівників, JIT та GC. У Go й Node.js звільнення тимчасових об’єктів може відбутися поза окремим виміром; це гіпотеза щодо відмінностей, а не встановлена причина. Профілювання фаз не проводилося. Для сильнішого статистичного висновку потрібні додаткові повтори та незалежні сесії.

Додаткові джерела

Rust threads: https://doc.rust-lang.org/std/thread/
Go concurrency and parallelism: https://go.dev/doc/effective_go#parallel
Python multiprocessing: https://docs.python.org/3/library/multiprocessing.html
Node.js workers: https://nodejs.org/api/worker_threads.html

11 Реалізація Cython + OpenMP

Додано окремий модуль із синтаксисом Python і типами Cython. Cython перетворює його на C, а GCC компілює з прапорцем -fopenmp. Писати C++ для цієї реалізації не потрібно. Це скомпільований модуль Python, а не звичайний інтерпретований цикл.

Як виконується сортування

1. Підготовка. Вхід — компактний масив int32. Паралельна версія створює його копію з p неперетинними діапазонами та p нульових гістограм uint64.
2. Паралельний підрахунок. Цикл prange із nogil=True розподіляє p блоків між потоками OpenMP. Кожна ітерація читає лише свій блок і змінює лише свій рядок гістограми.
3. Очікування. Неявний бар’єр наприкінці prange завершується після готовності всіх гістограм.
4. Завершення. Головний потік послідовно додає гістограми й відновлює відсортований масив.

Спільна пам’ять і приватні дані — різні поняття

Реалізація	Адресний простір та обмін
Python, процеси	Окремі адресні простори; серіалізація копій через IPC
C++, Rust, Go	Спільний адресний простір; власні блоки й гістограми задач
Node.js	Окремі JS-ізоляти; передавання власності на копії ArrayBuffer, без SharedArrayBuffer
Cython + OpenMP	Спільний адресний простір; неперетинні блоки та окремі рядки гістограм

У Go передавання slice через канал не копіює весь його буфер: канал передає дескриптор. Копіювання блоку в нашій програмі виконується явно до надсилання задачі. Тому називати Go або OpenMP реалізаціями «без спільної пам’яті» некоректно: у них немає спільних одночасних записів у лічильники, але пам’ять процесу спільна.

Параметри та перевірка

workers = 0 задає послідовний Cython-варіант; workers = 1, 2, 4 — OpenMP. Фактичний розмір команди записується в CSV: у цій сесії отримано 1, 2 та 4 потоки. Динамічне зменшення команди вимкнено, але зовнішні обмеження середовища все одно можуть впливати на кількість потоків.

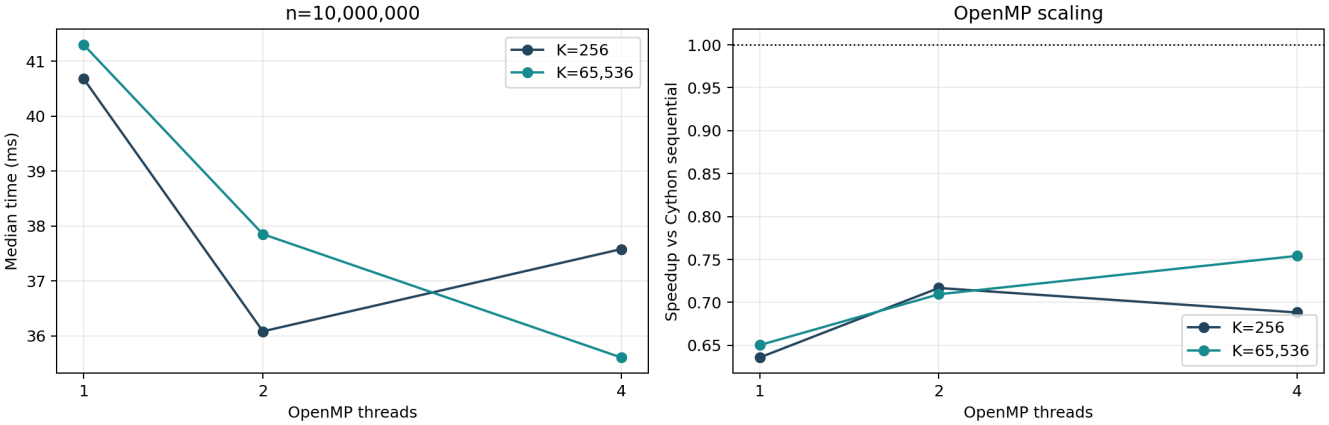
Перевірено 60 наборів у чотирьох конфігураціях, шість некоректних входів і крайні значення int32. Після кожного вимірювання весь вихід звірено з еталонним сортуванням. Вхід не змінюється. Виклик виконується з головного потоку; вхідний буфер не можна одночасно змінювати.

12 Результати Cython + OpenMP

Окрема серія на тій самій машині: ті самі вісім наборів, п’ять повторів, 160 вимірювань. SHA-256 усіх входів збіглися з базовим порівнянням п’яти мов. Збірка: Cython 3.3.0, GCC 13.3, -O3 -fopenmp; модуль зв’язано з libgomp.

K	Послідовно	p = 1	p = 2	p = 4	S_4
256	25.866	40.679	36.084	37.581	0.69
65 536	26.857	41.299	37.853	35.608	0.75

n = 10 000 000; час у мілісекундах, медіана п’яти повторів. S_4 відносно послідовної версії Cython, а не інтерпретованого Python.



Що входить у час

Виділення пам’яті, копіювання входу в паралельній версії, підрахунок, додавання гістограм, запис виходу та звільнення локальних буферів. Генерація входу, побудова еталона, перевірка, компіляція та запуск головної програми виключені. Перед вимірюванням кожну конфігурацію прогріто один раз.

Кожен паралельний виклик відкриває область OpenMP; середовище може зберігати потоки ОС між викликами. Тому цю серію не позначено cold: вона не відтворює створення та завершення наших пулів. Абсолютні часи різних серій залежать від поточного стану машини.

Висновок

Для n = 10 000 000, K = 256 паралельний варіант із чотирма потоками виконався за 37.581 мс проти 25.866 мс послідовно; S_4 = 0.69, тобто паралельний варіант у 1.45 раза повільніший. Алгоритм зберігає послідовні етапи копіювання, об’єднання та запису виходу, тому збільшення кількості потоків не гарантує пропорційного прискорення.

Код та інструкції: https://github.com/blcktqq/parallel-counting-sort-benchmark
Документація Cython/OpenMP: https://cython.readthedocs.io/en/stable/src/userguide/parallelism.html